LAPORAN PRAKTIKUM 7

PEMOGRAMAN BERBASIS WEB CP

“Effect Custom Hook dan Router”

Dosen Pengampu :

Dr. Puspaningtyas Sanjoyo Adi S.T., M.T.



Oleh :

Fransisca Noviana Prina Ramiro Soares

255314083

PROGRAM STUDI INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS SANATA DHARMA

YOGYAKARTA

2026

A. Tujuan

1. Memahami perbedaan render, event handler, dan Effect, termasuk dependency dan cleanup.
2. Membuat custom hook (usePageTitle dan usePersistentFavorites) dan memahami bahwa memakai hook yang sama berulang tidak otomatis berbagi state.
3. Membuat aplikasi SPA dengan React Router: route biasa, route dinamis (/products/:id), navigasi aktif, dan halaman tidak ditemukan.
4. Menentukan pemilik state bersama (Provider) dan menyinkronkan state dengan localStorage dan URL.
5. Menguji aplikasi secara fungsional dan menjelaskan hasilnya.

B. Masalah atau Soal

1. Menambahkan kategori pada produk dan filter melalui query parameter category, dengan q tetap dipertahankan.
2. Menambahkan tombol Hapus semua favorit yang membuat badge, kartu, dan localStorage tetap konsisten setelah aksi dan reload.
3. Menambahkan halaman Tentang dengan judul tab dari custom hook.
4. Menjelaskan mengapa pemanggilan hook berulang tidak berbagi state, dan menunjukkan Provider sebagai pemilik state.
5. Melengkapi 12 kasus uji inti ditambah 2 kasus tambahan (filter kategori dan hapus semua).

C. Hasil dan Analisis Jawaban

Analisis tiap soal, kemudian dilanjutkan dengan hasil pengujian kasus.

Soal 1: Kategori dan filter category

Pada program ini, setiap produk memiliki kategori masing-masing, yaitu Buku atau Aksesori. Komponen Catalog digunakan untuk menampilkan produk berdasarkan pencarian nama dan kategori yang dipilih. Jadi, ketika pengguna mengetik nama produk dan memilih kategori, sistem akan menampilkan produk yang sesuai dengan kedua pilihan tersebut. Jika kategori tidak dipilih, semua kategori tetap ditampilkan. Fungsi updateParam digunakan untuk memperbarui parameter pada URL tanpa menghapus parameter lainnya. Dengan begitu,

ketika kategori diubah, kata kunci pencarian tetap tersimpan, begitu juga sebaliknya. Nilai pencarian dan kategori juga langsung diambil dari URL sehingga tidak perlu membuat state tambahan. Hasilnya, pengguna dapat mencari dan menyaring produk dengan lebih mudah, serta melihat kategori pada dropdown dan setiap kartu produk.

```
function updateParam(key, value, replace = true) {
  const next = new URLSearchParams(params);
  if (value) next.set(key, value);
  else next.delete(key);
  setParams(next, { replace });
}
```

Soal 2: Tombol Hapus semua favorit

Pada bagian ini, ditambahkan fungsi clear() pada usePersistentFavorites untuk menghapus semua produk yang sudah dimasukkan ke daftar favorit. Fungsi ini menggunakan setIds([]) sehingga seluruh data favorit menjadi kosong. Data pada localStorage juga otomatis diperbarui karena sudah menggunakan useEffect yang memantau perubahan ids. Badge pada navigasi dan halaman Favorit akan ikut berubah secara otomatis karena keduanya mengambil data dari Context yang sama. Tombol hapus semua juga dibuat tidak aktif ketika belum ada produk favorit, sehingga tidak bisa ditekan jika daftar masih kosong.

```
function clear() {
  setIds([]);
}
return { ids, toggle, clear, storageError };
```

Soal 3: Halaman Tentang dan judul tab

Pada bagian ini, ditambahkan halaman baru yaitu /about dengan nama halaman Tentang. Menu Tentang juga ditambahkan pada navigasi menggunakan NavLink agar pengguna bisa membuka halaman tersebut. Komponen About menggunakan custom hook usePageTitle("Tentang Aplikasi") untuk mengubah judul pada tab browser sesuai halaman yang sedang dibuka. Ketika pengguna meninggalkan halaman Tentang, judul browser akan dikembalikan seperti sebelumnya. Dari hasil pengujian, menu Tentang sudah berhasil muncul pada navigasi di setiap halaman.

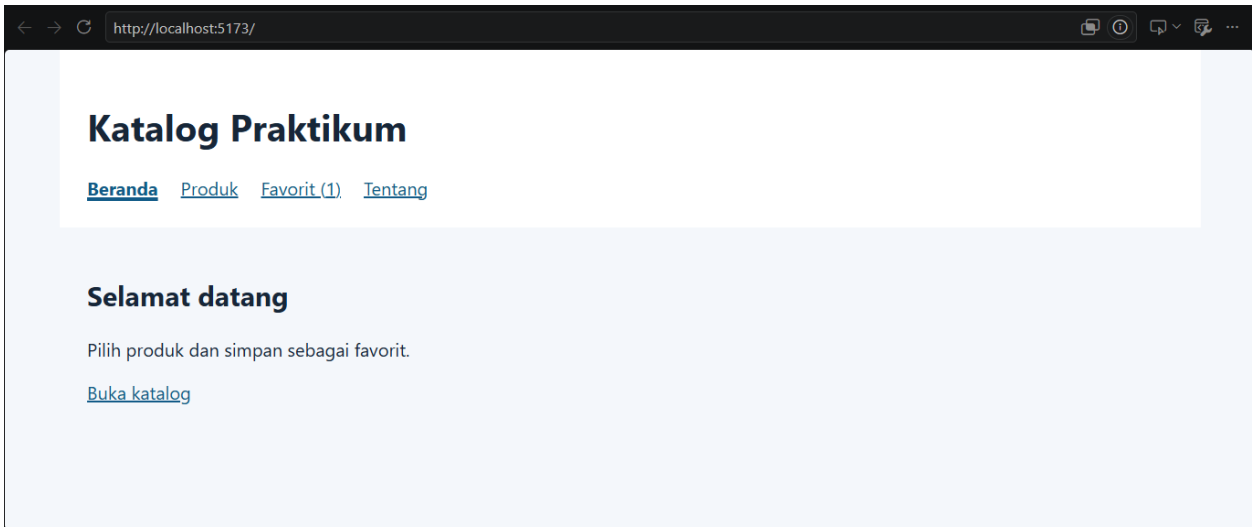
Soal 4: Hook berulang tidak berbagi state

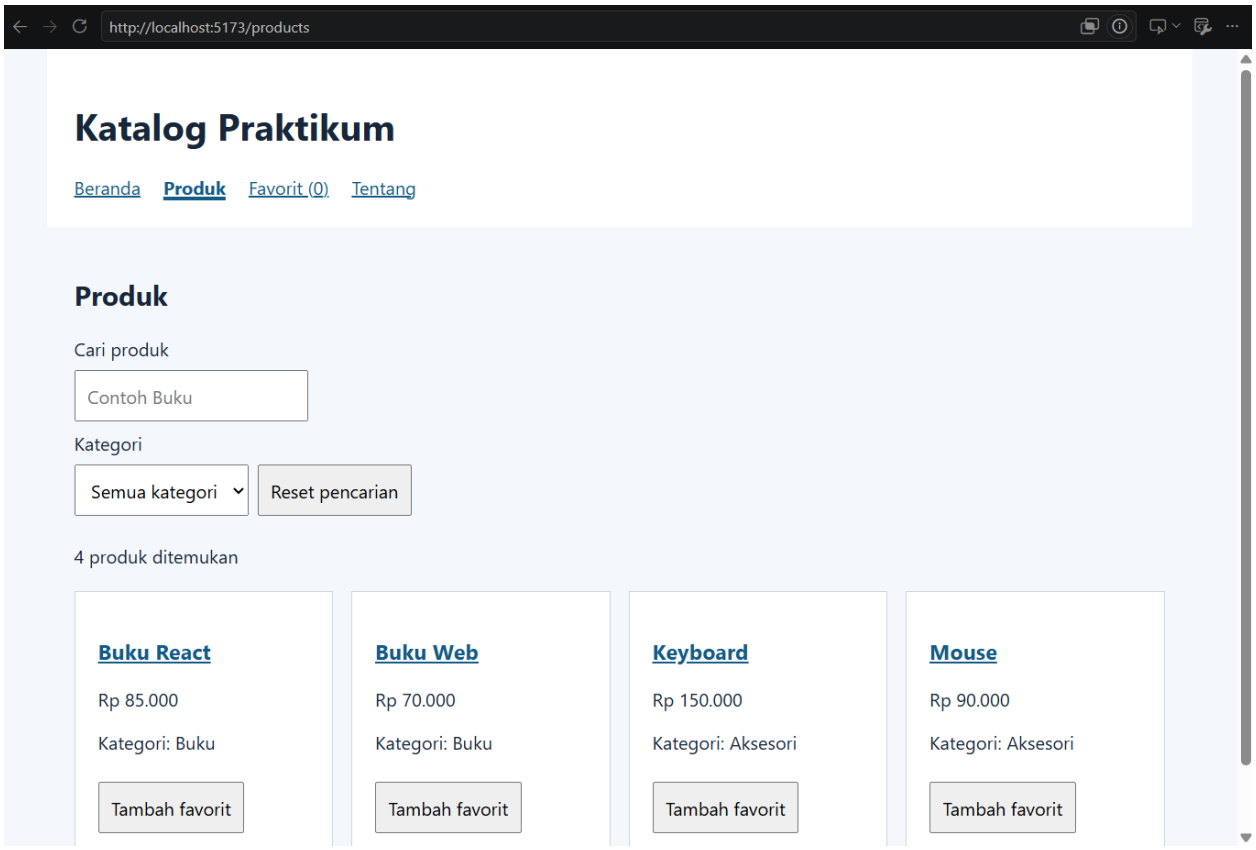
Pada bagian ini, dijelaskan bahwa setiap komponen yang menggunakan useState akan memiliki state masing-masing. Jika halaman Catalog dan Favorites sama-sama memanggil usePersistentFavorites(), maka data favorit keduanya bisa berbeda dan tidak otomatis saling terhubung. Untuk mengatasi hal tersebut, dibuat FavoritesProvider sebagai tempat penyimpanan state favorit yang digunakan bersama oleh semua komponen. Jadi, hanya Provider yang memanggil usePersistentFavorites(), sedangkan komponen lain mengambil data melalui useFavorites(). Provider kemudian dipasang di main.jsx dan membungkus komponen App agar data favorit tetap sama meskipun pengguna berpindah halaman.

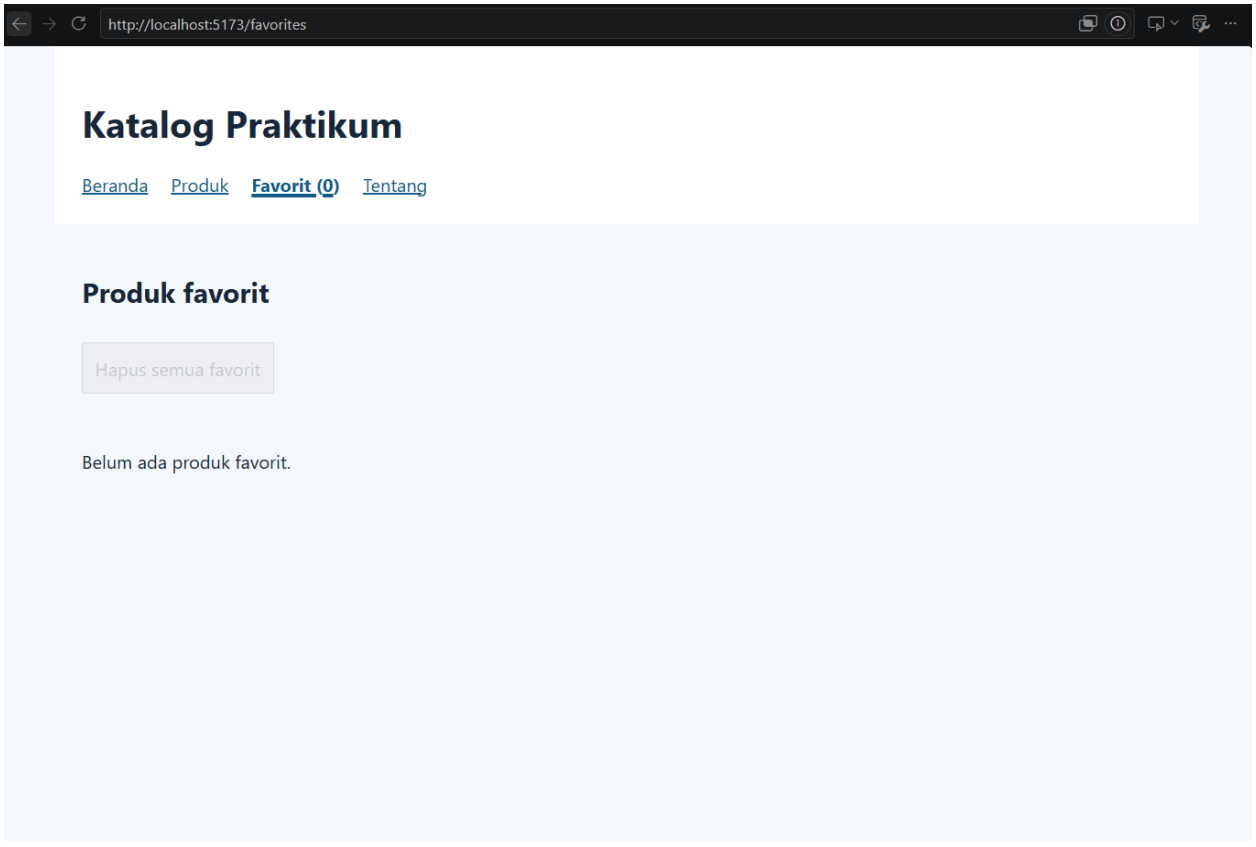
```
// src/context/FavoritesProvider.jsx
export function FavoritesProvider({ children }) {
  const value = usePersistentFavorites(); // satu-satunya pemilik state
  return (
    <FavoritesContext.Provider value={value}>
      {children}
    </FavoritesContext.Provider>
  );
}

// src/main.jsx
<BrowserRouter>
  <FavoritesProvider>
    <App />
  </FavoritesProvider>
</BrowserRouter>
```

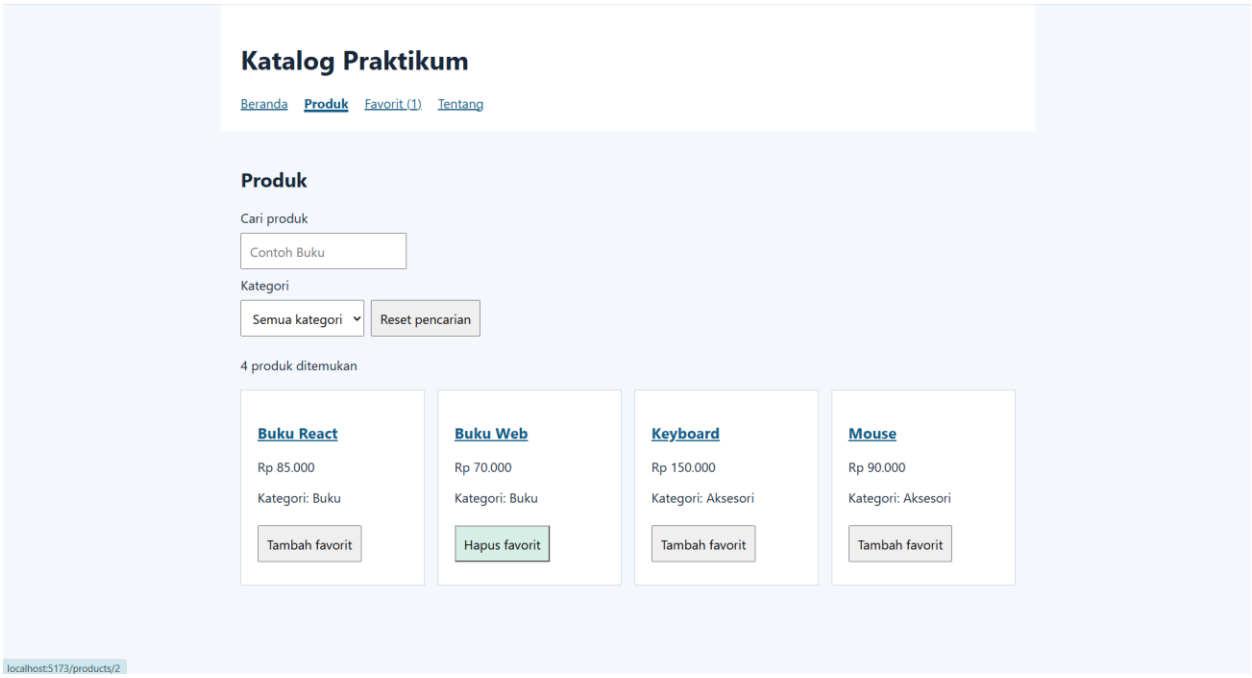
Output




http://localhost:5173/products
Katalog Praktikum
Beranda Produk Favorit (0) Tentang
Produk
Cari produk
Contoh Buku
Kategori
Semua kategori
Reset pencarian
4 produk ditemukan
Buku React
Rp 85.000
Kategori: Buku
Tambah favorit
Buku Web
Rp 70.000
Kategori: Buku
Tambah favorit
Keyboard
Rp 150.000
Kategori: Aksesori
Tambah favorit
Mouse
Rp 90.000
Kategori: Aksesori
Tambah favorit


http://localhost:5173/favorites
Katalog Praktikum
Beranda Produk Favorit (0) Tentang
Produk favorit
Hapus semua favorit
Belum ada produk favorit.

Pengujian kasus

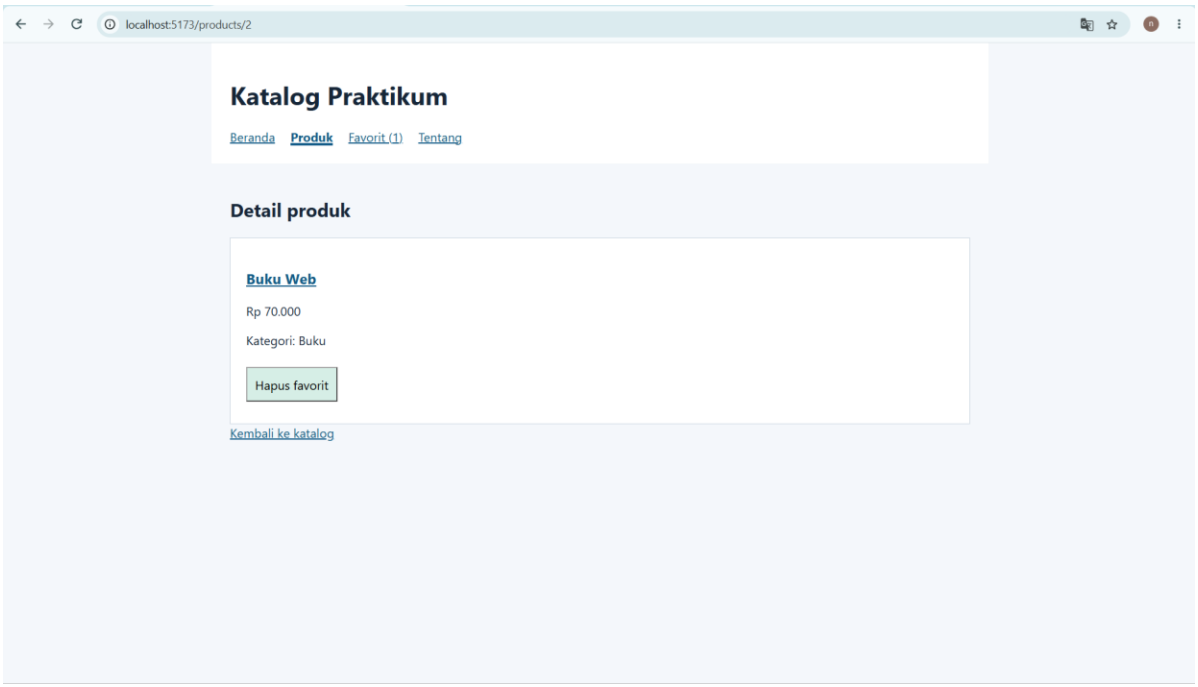


Menambahkan Produk dari Katalog

Hasil: Setelah tombol Tambah Favorit pada produk Buku Web ditekan, tombol berubah menjadi Hapus Favorit dengan warna hijau. Menu Favorit juga berubah menjadi Favorit (1). Pada halaman produk, terdapat 4 produk yang ditampilkan beserta kategori masing-masing.

Analisis: Saat tombol Tambah Favorit ditekan, fungsi toggle akan memperbarui data favorit di FavoritesProvider. Badge pada menu dan tombol pada kartu produk menggunakan data Context yang sama, sehingga perubahan terjadi secara otomatis dan bersamaan tanpa perlu membuat state tambahan.

Status: Lulus.

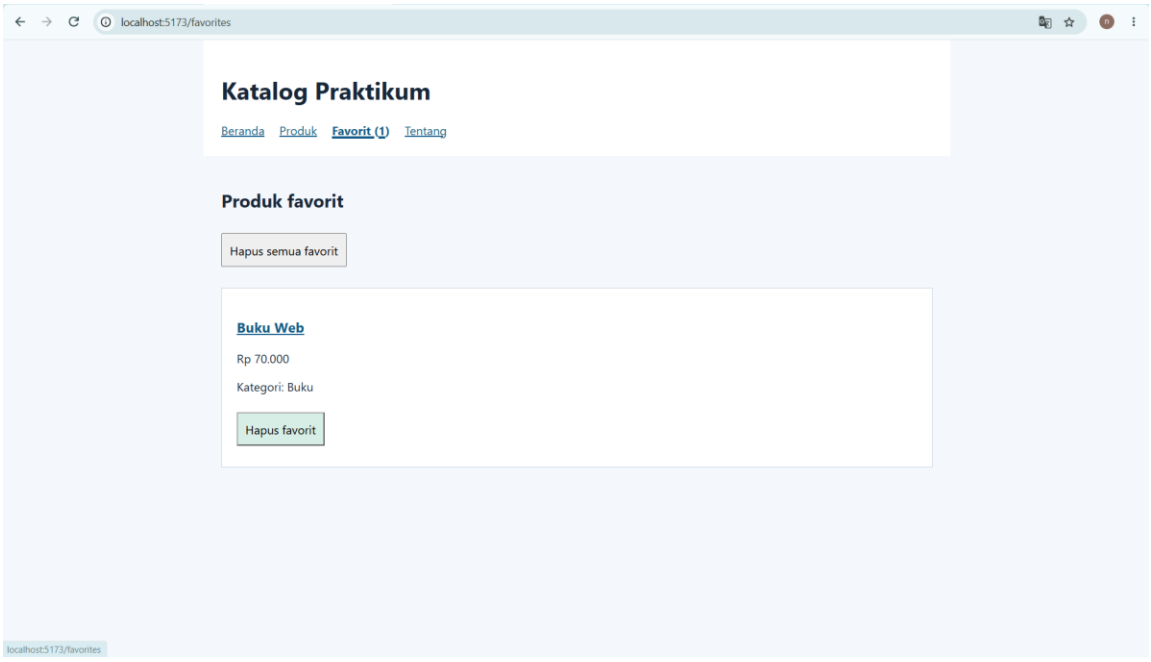


Detail Produk Tetap Ditandai

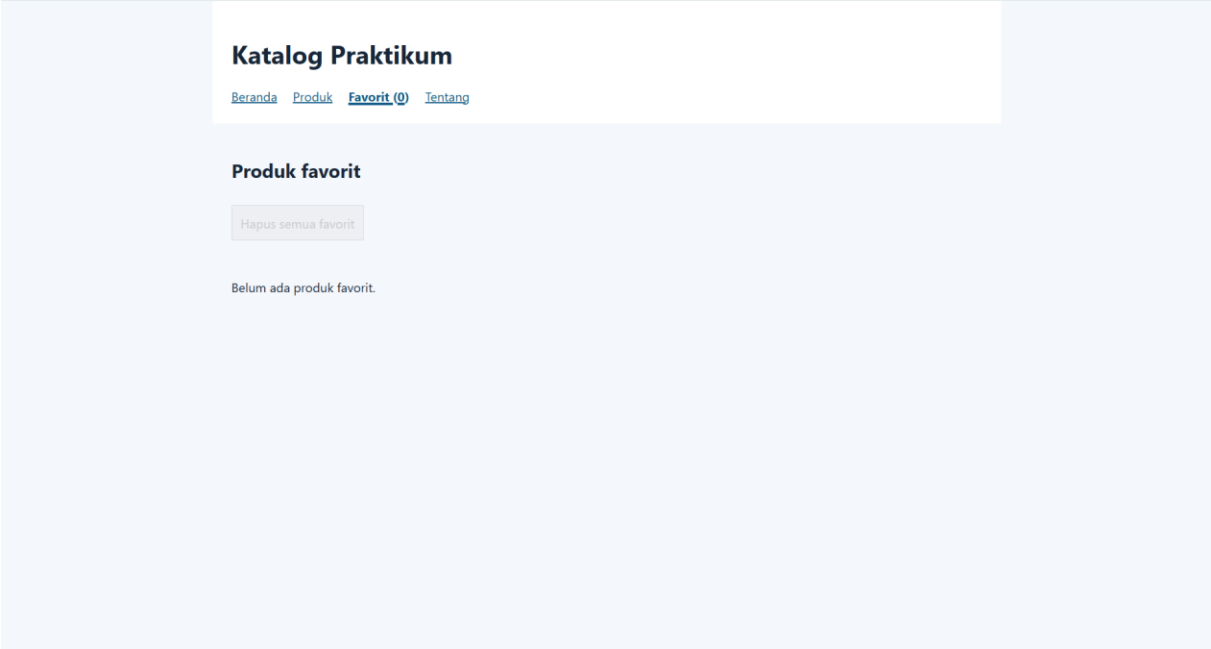
Hasil: Pada halaman /products/2, produk Buku Web berhasil ditampilkan dengan tombol Hapus Favorit dan badge pada menu menunjukkan Favorit (1).

Analisis: Halaman detail produk menggunakan ProductCard dan Context yang sama dengan halaman katalog. Oleh karena itu, status favorit tetap konsisten meskipun pengguna berpindah halaman.

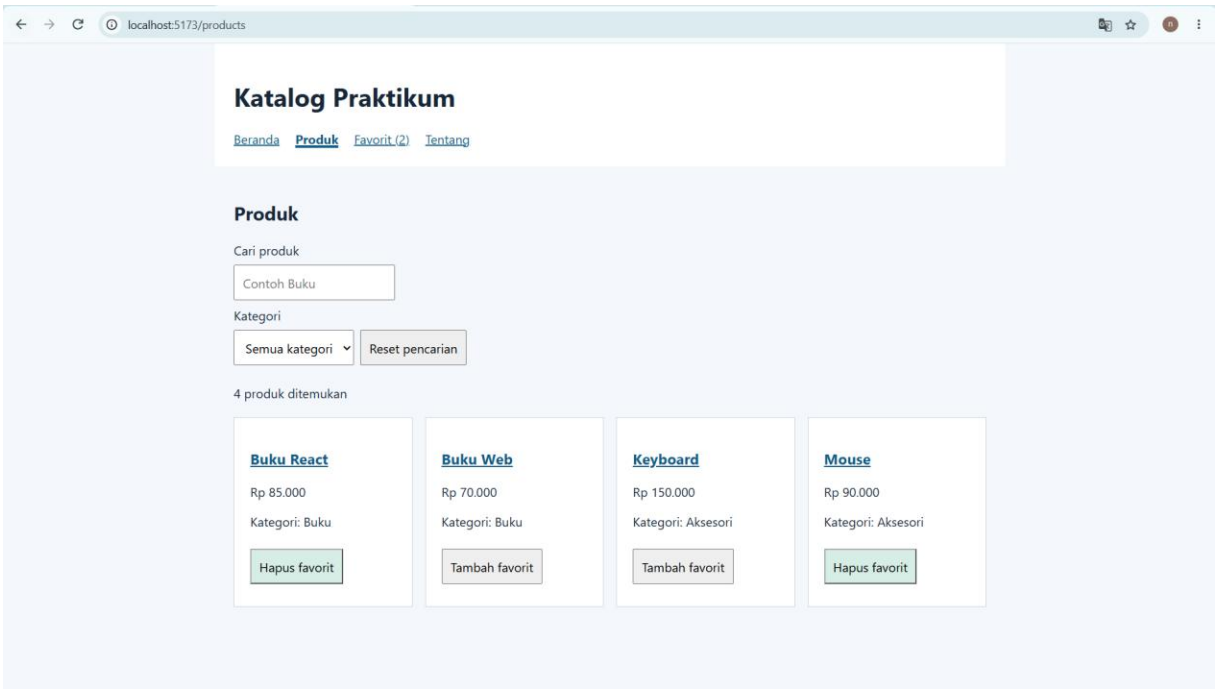
Status: Lulus.



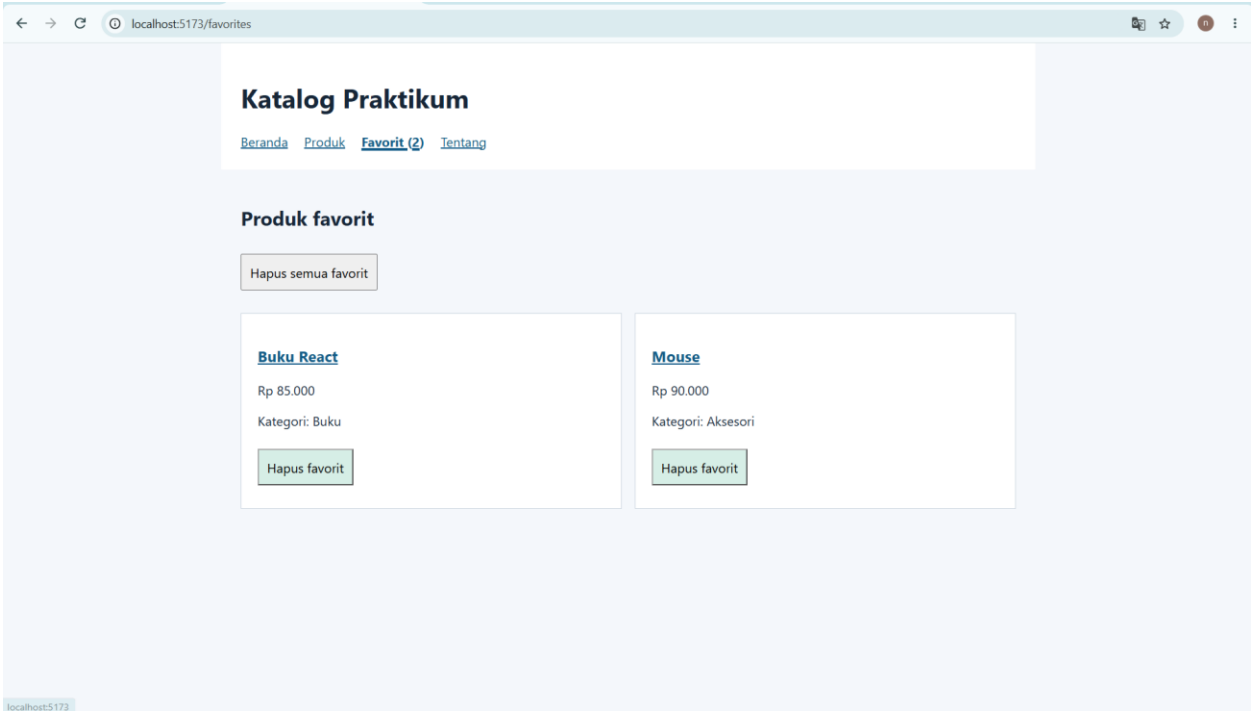
Hasil: Halaman /favorites menampilkan Buku Web dengan badge Favorit (1). Analisis: Halaman Favorit menyaring PRODUCTS berdasarkan ids, sehingga daftar yang tampil selalu turunan dari state bersama. Status Lulus.



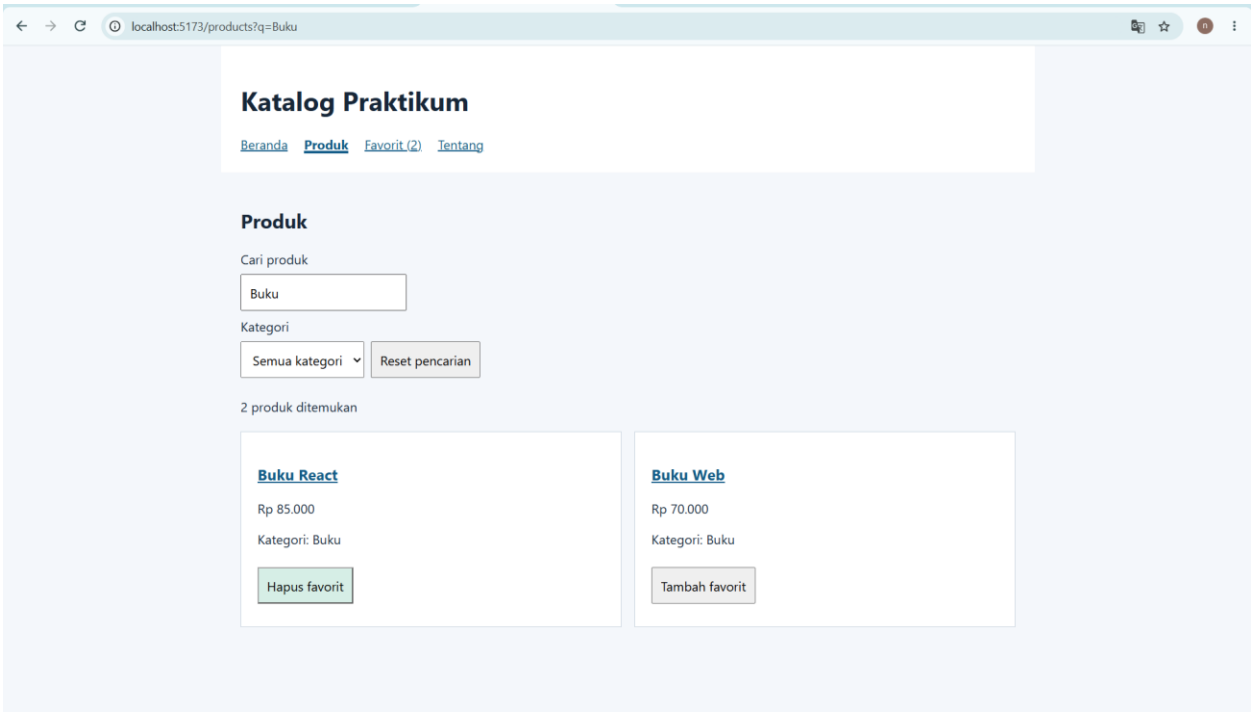
Hapus produk dari favorit. Hasil: Setelah favorit dihapus, kartu hilang, menu menjadi Favorit (0), muncul pesan “Belum ada produk favorit.”, dan tombol Hapus semua favorit nonaktif. Analisis: Daftar favorit dihitung saat render dari ids sehingga kartu hilang otomatis tanpa Effect tambahan. Status: Lulus.



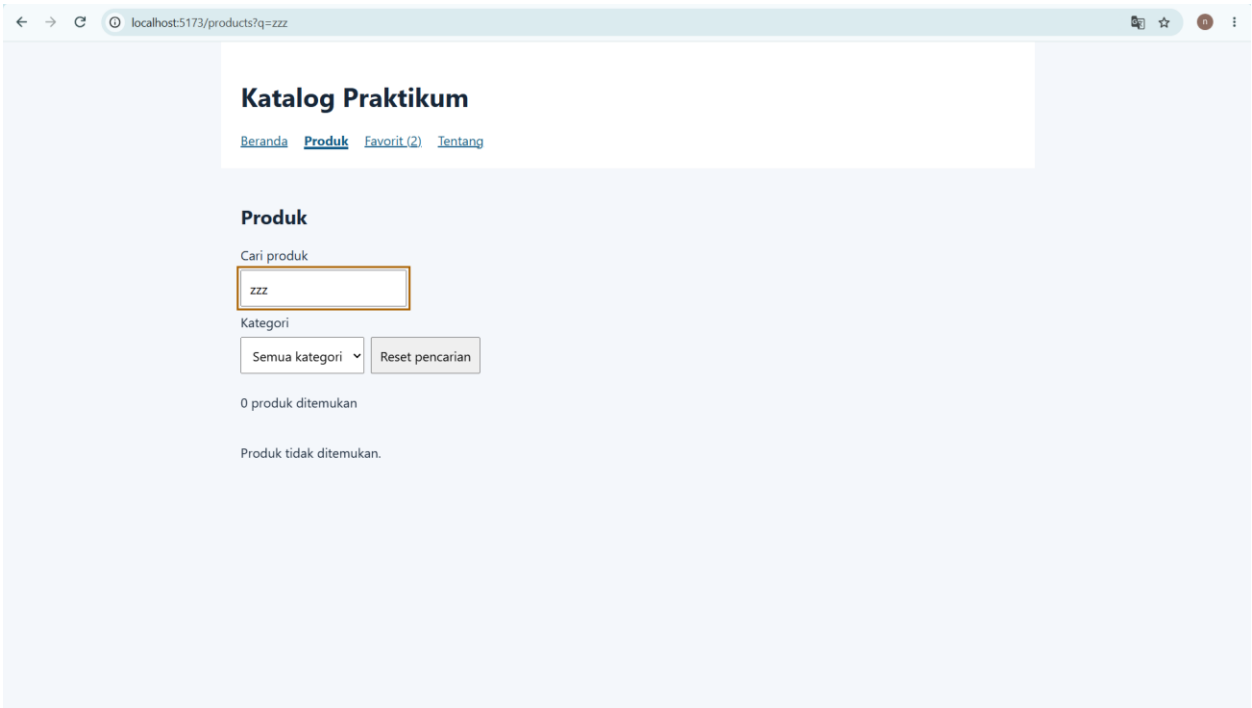
Dua favorit tersimpan. Hasil: Halaman Produk menandai Buku React dan Mouse dengan Hapus favorit, dan menu menunjukkan Favorit (2). Analisis: Nilai awal ids dibaca dari localStorage oleh readIds sebagai lazy initializer, sedangkan Effect dengan dependency [ids] menyimpan perubahan berikutnya.



Hasil: Halaman /favorites menampilkan dua kartu, yaitu Buku React dan Mouse, dengan badge Favorit (2). Analisis: Data favorit konsisten antara halaman Produk dan Favorit. Status Lulus.

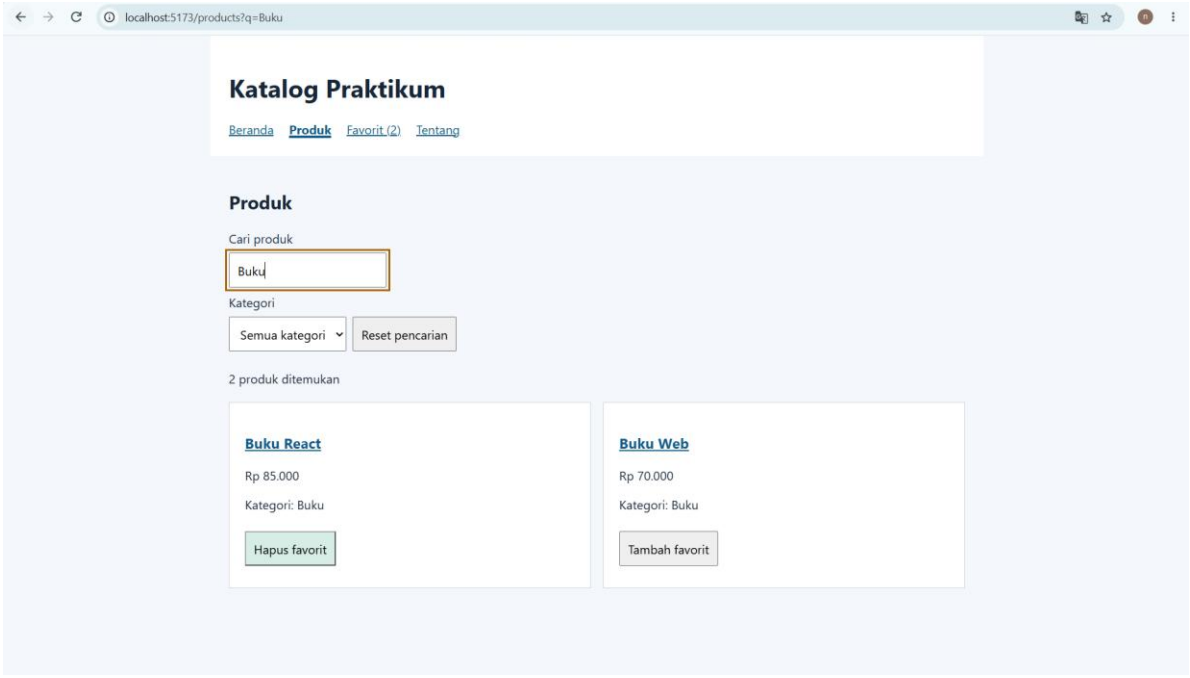


Pencarian dari URL. Hasil: URL /products?q=Buku menampilkan kolom cari berisi "Buku" dan 2 produk (Buku React dan Buku Web). Analisis: Nilai q dibaca langsung dari URL dengan useSearchParams dan menjadi value input, sehingga URL yang dibagikan menghasilkan tampilan yang sama. Status: Lulus.

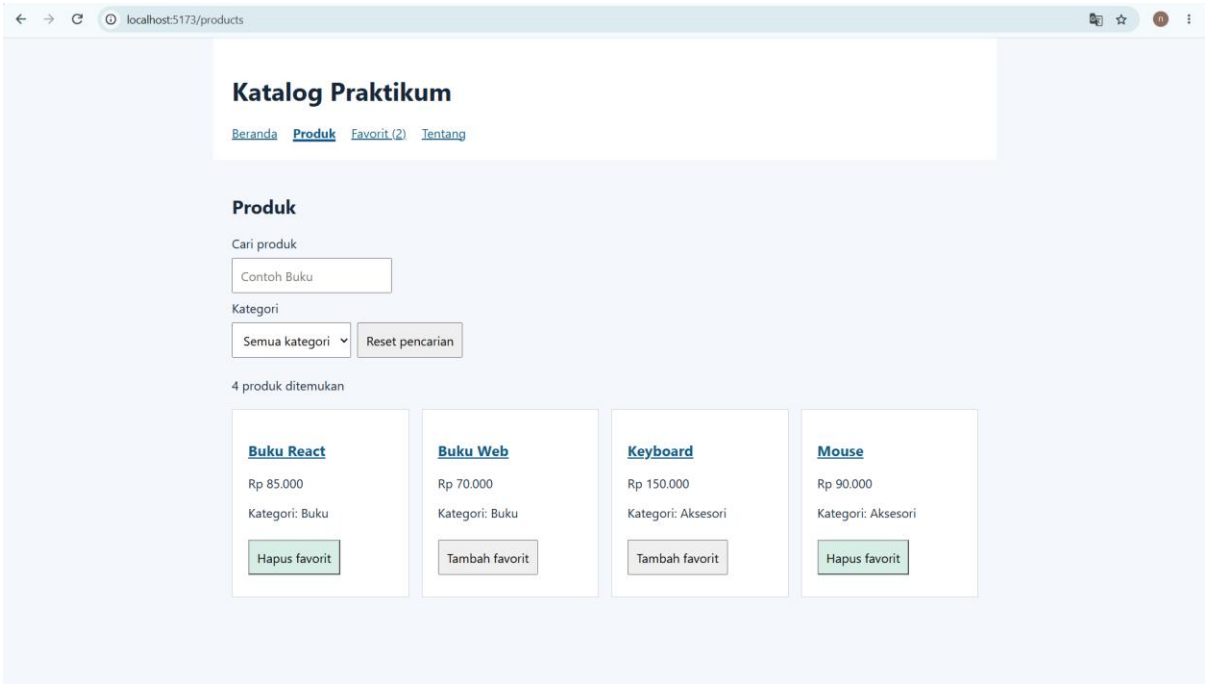


Kata Pencarian Tidak Cocok Hasil: Pencarian "zzz" menampilkan 0 produk ditemukan dan pesan "Produk tidak ditemukan."

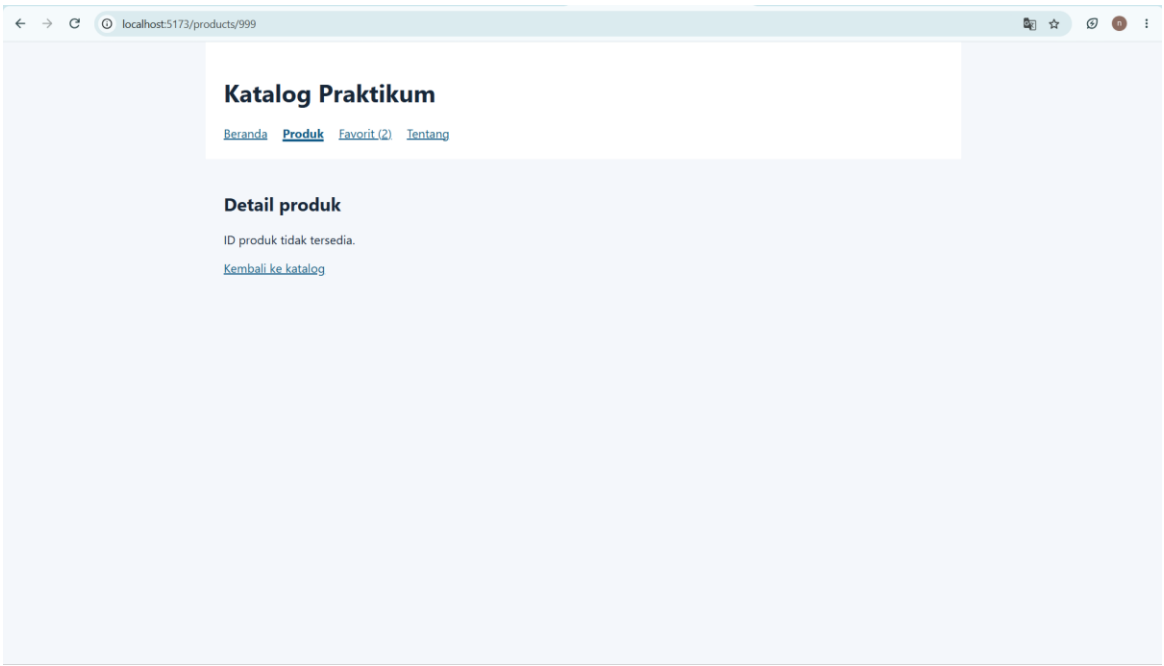
Analisis: Tidak ada produk yang cocok dengan kata pencarian, sehingga sistem menampilkan pesan tersebut tanpa error. Status: Lulus.



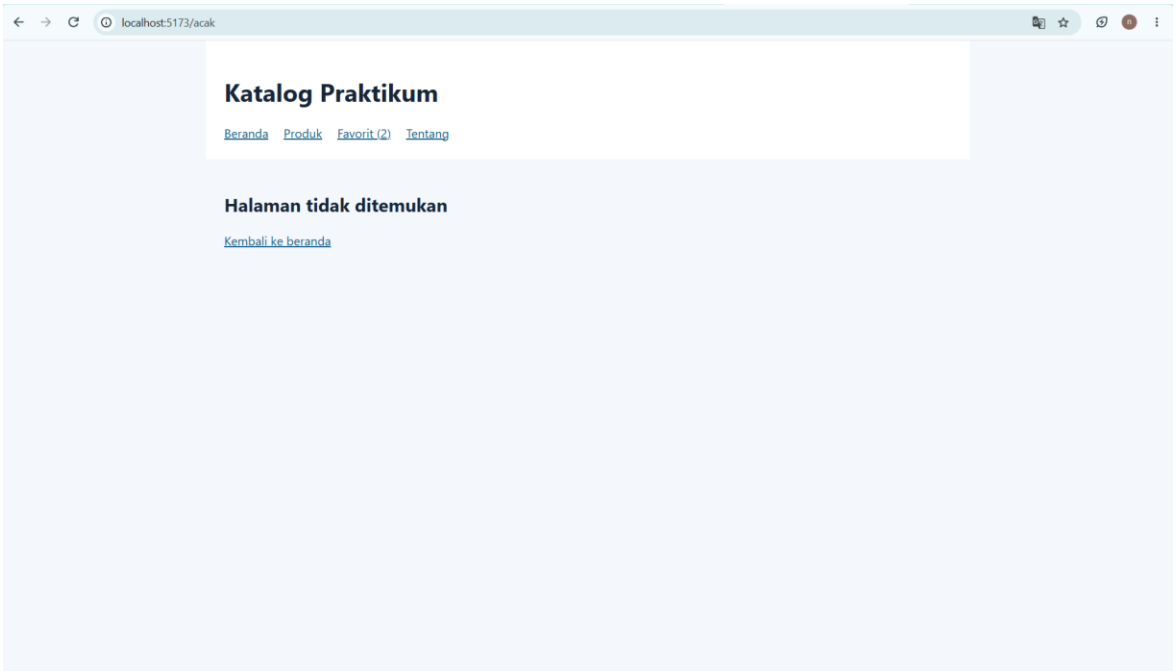
Reset pencarian



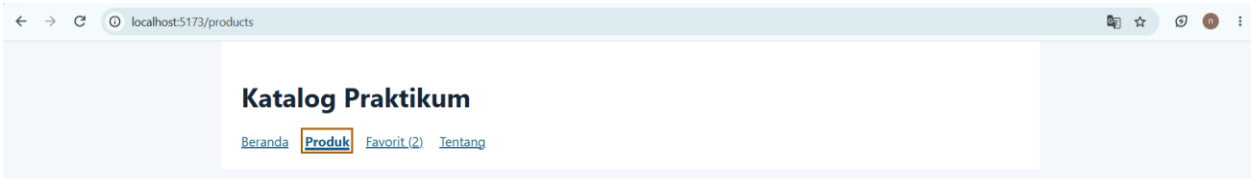
Reset pencarian. Hasil: Pencarian “Buku” menghasilkan 2 produk Setelah tombol Reset pencarian ditekan, URL kembali ke /products, kolom cari kosong, dan 4 produk tampil. Analisis: Reset memakai replace: false sehingga membuat entri riwayat baru, dan input mengikuti URL karena tidak ada state pencarian lokal.



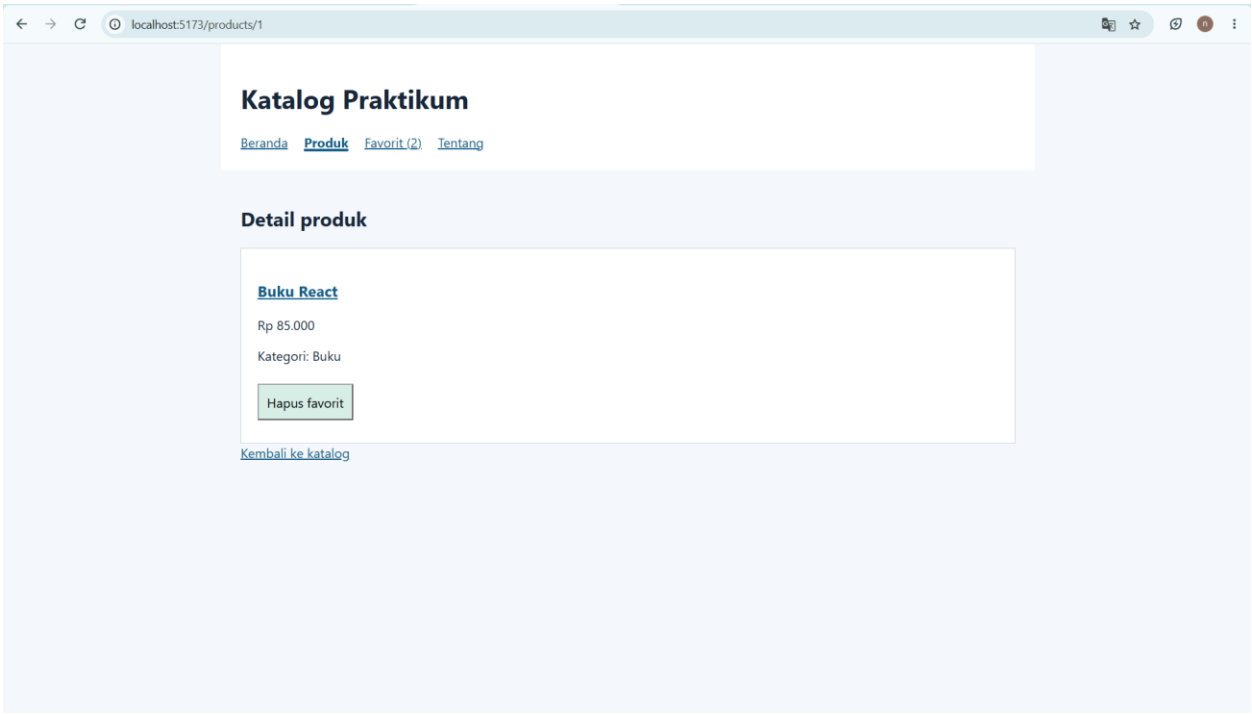
ID produk tidak tersedia. Hasil: URL /products/999 menampilkan “ID produk tidak tersedia.” beserta tautan Kembali ke katalog. Analisis: useParams mengembalikan string dan dibandingkan dengan String(item.id). Karena ID 999 tidak ada, komponen Detail menampilkan pesan tersebut.



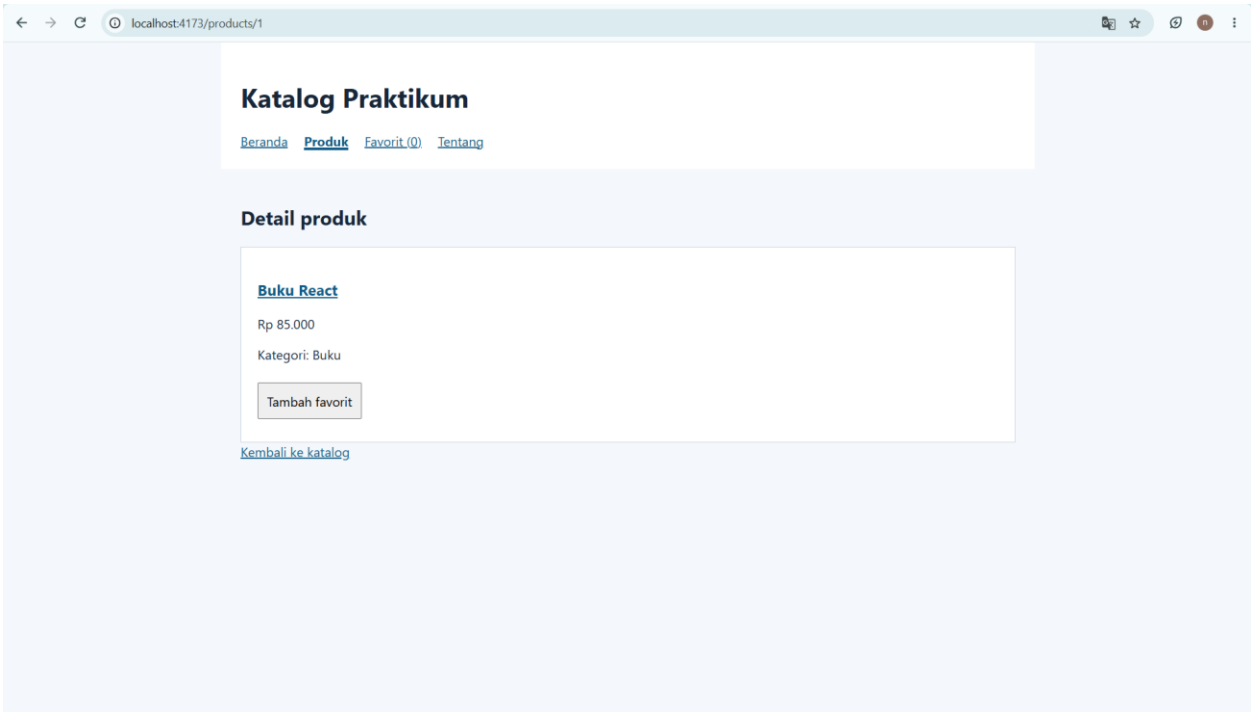
Halaman Tidak Ditemukan Hasil: URL /acak menampilkan pesan "Halaman tidak ditemukan" dan tautan Kembali ke Beranda.
Analisis: Route wildcard * digunakan untuk menangani halaman yang tidak tersedia.
Status: Lulus

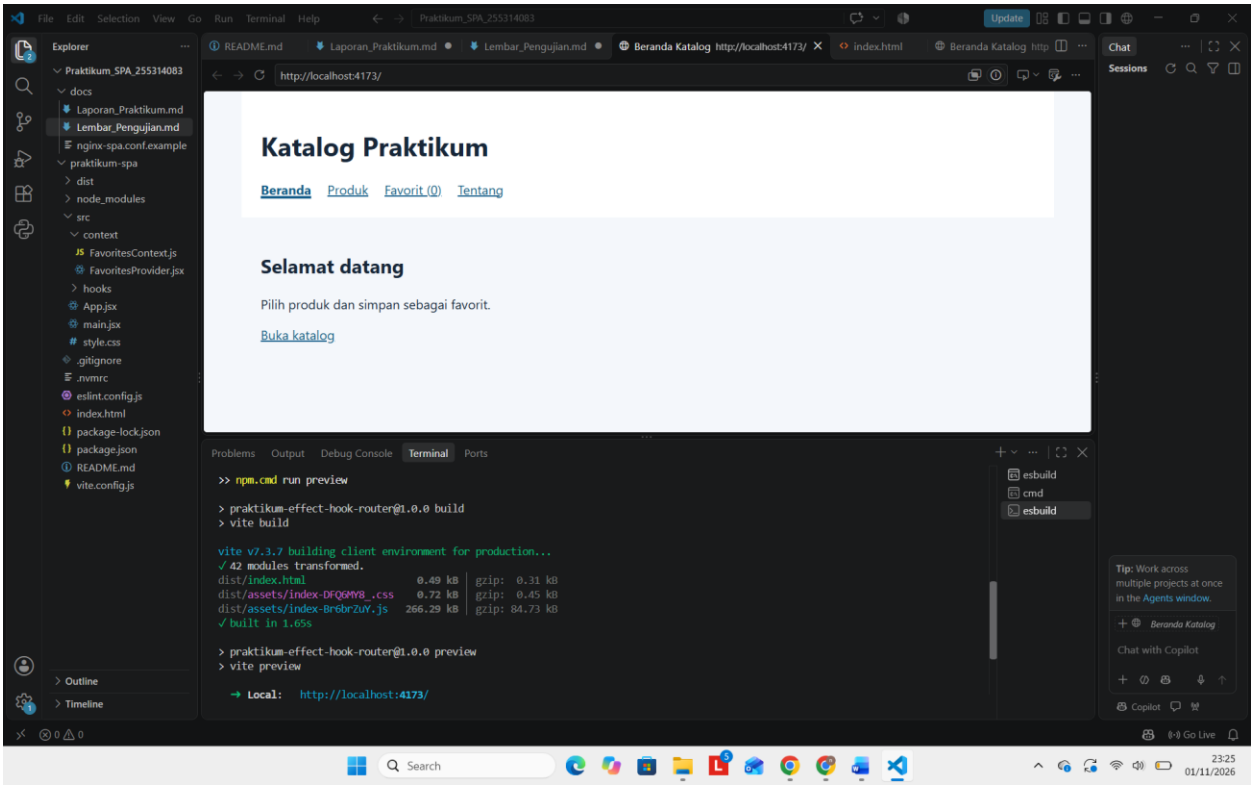


Navigasi dengan Keyboard
Hasil: Saat tombol Tab ditekan, fokus berpindah ke tautan Produk dengan tanda garis oranye. Analisis: CSS :focus-visible membantu menunjukkan posisi fokus sehingga tautan dan tombol bisa digunakan tanpa mouse.
Status: Lulus.

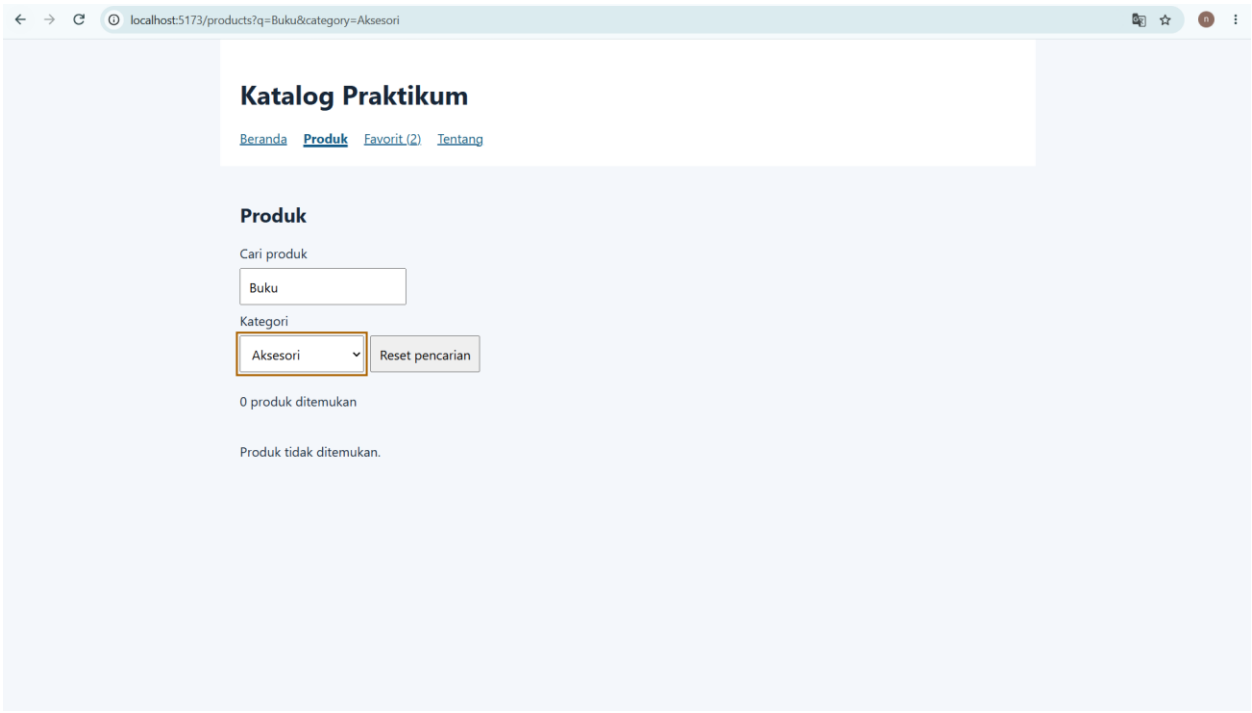


(uji terpisah, server dev): Buka /products/1 langsung. Hasil: Alamat localhost:5173/products/1 membuka halaman Detail produk Buku React. Analisis: Server Vite mengembalikan index.html untuk route aplikasi, sehingga BrowserRouter dapat menampilkan halaman detail.

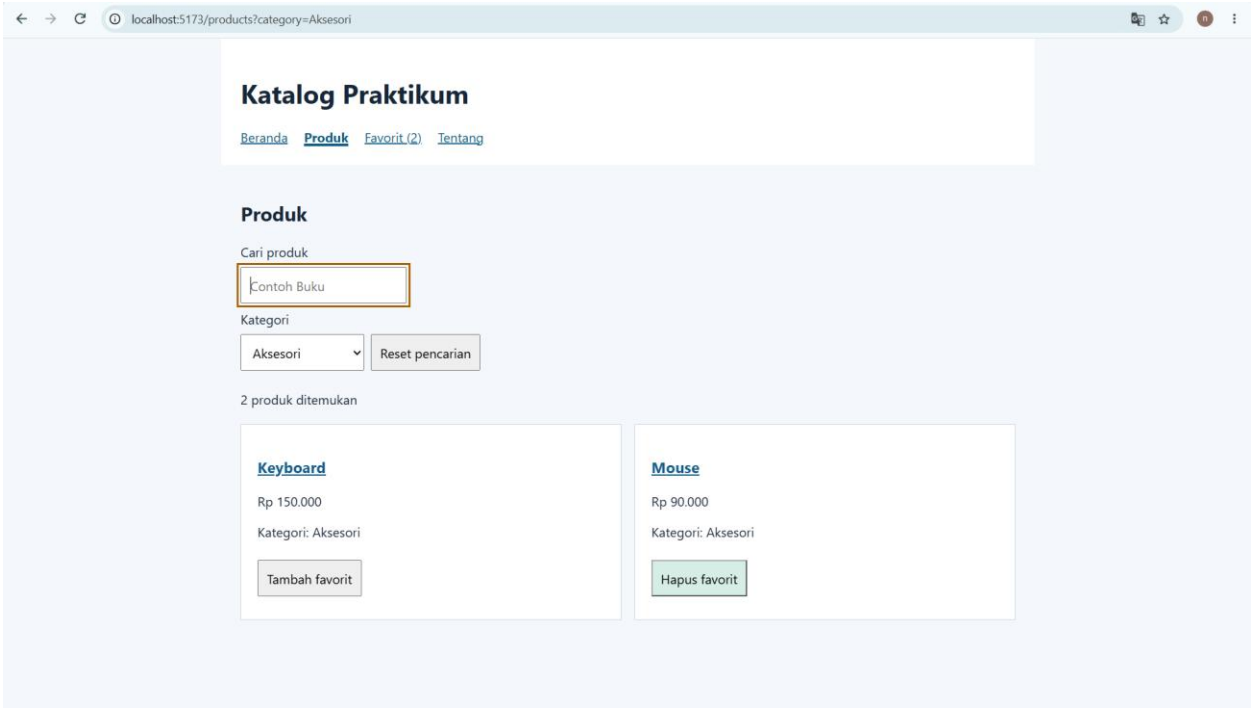


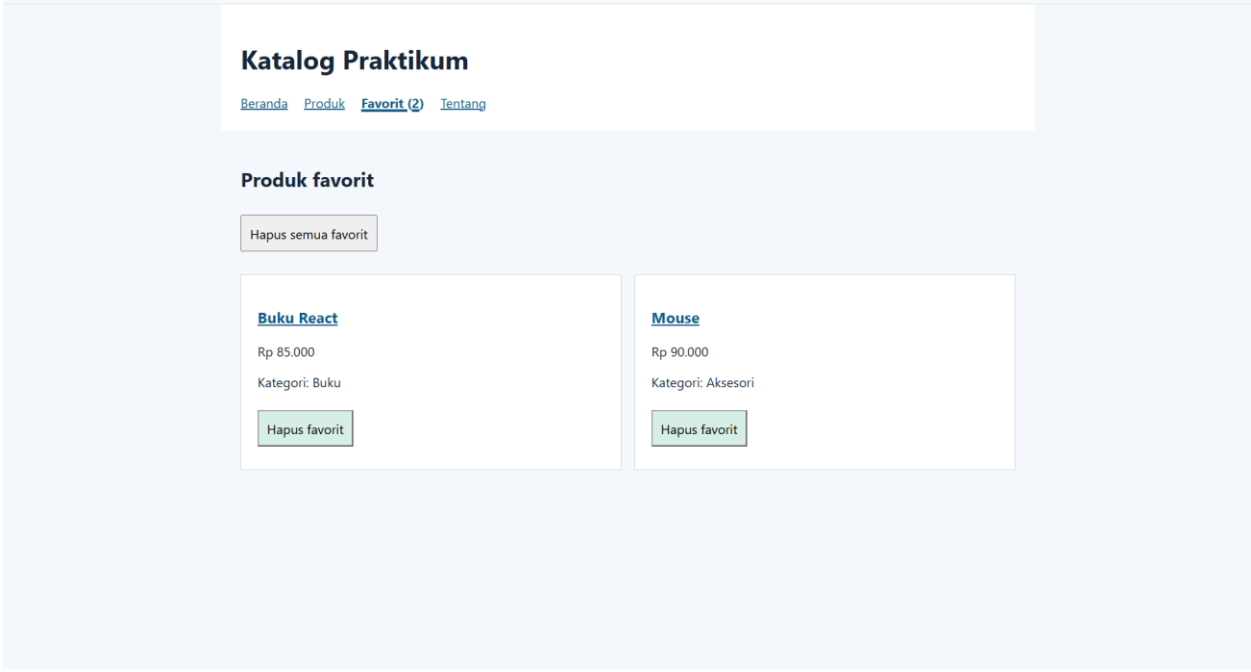


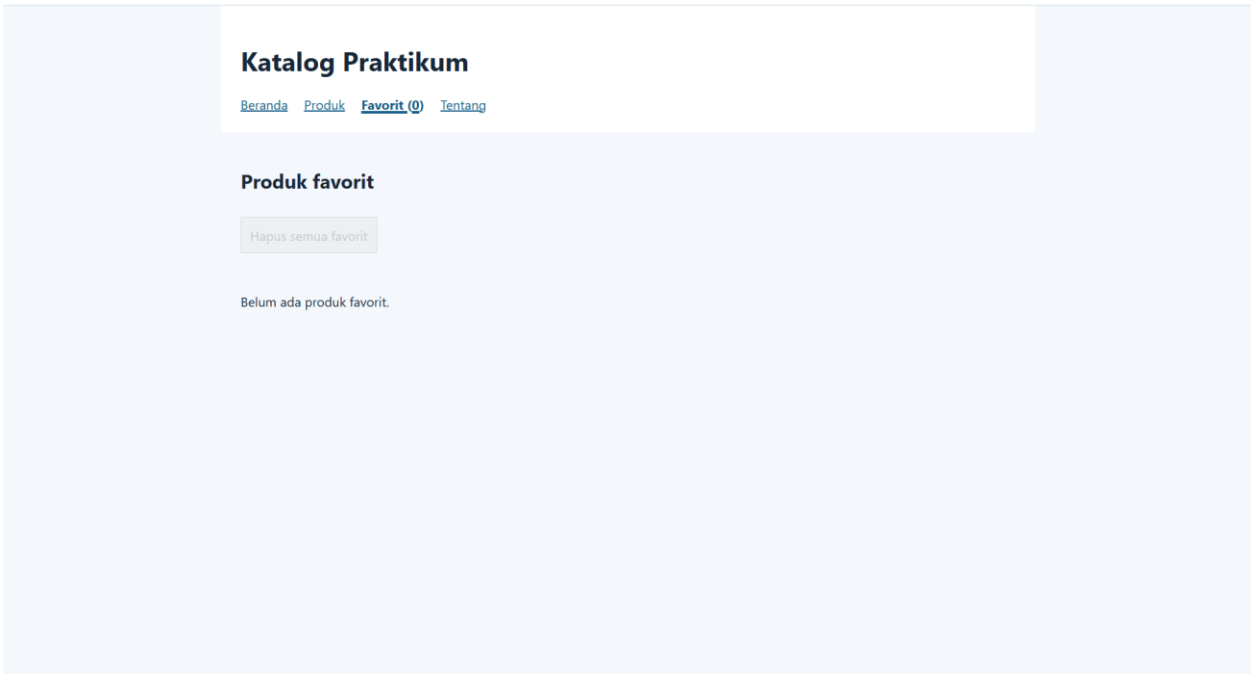
Hasil build dan preview: npm run build berhasil mentransformasi 42 modul dan selesai dalam 1,65 detik, kemudian npm run preview berjalan di http://localhost:4173/. Beranda tampil normal dengan menu Beranda, Produk, Favorit, dan Tentang.



Buku dan kategori Aksesori menghasilkan URL ?q=Buku&category=Aksesori







“Setelah Hapus semua favorit, badge menjadi 0, daftar kosong, dan tetap kosong setelah reload. Local Storage berisi [].”

D. Kesimpulan dan Refleksi

Kesimpulan

Aplikasi SPA katalog berhasil dibuat dengan fitur halaman, pencarian, kategori, dan favorit. Data favorit dikelola melalui FavoritesProvider agar tetap sama di setiap halaman. Pencarian dan kategori disimpan di URL sehingga bisa dibagikan dan mengikuti tombol Back dan Forward. Sementara itu, data favorit disimpan menggunakan localStorage agar tidak hilang saat halaman di-refresh. Namun, sinkronisasi antar-tab dan antarperangkat belum tersedia.

Refleksi

1. Mengapa filtered dan jumlah favorit tidak perlu menggunakan useEffect?

Karena keduanya bisa dihitung langsung dari data yang sudah tersedia, seperti produk, pencarian, kategori, dan jumlah favorit. Jadi, tidak perlu membuat state tambahan.

2. Apa yang terjadi jika Provider diletakkan di dalam komponen halaman?

Data favorit bisa hilang ketika pengguna berpindah halaman karena state dibuat ulang. Oleh karena itu, Provider diletakkan di atas App agar bisa digunakan oleh semua halaman.

3. Apa perbedaan URL state, state React, dan data persisten?

URL state menyimpan data pada alamat URL sehingga bisa dibagikan. State React menyimpan data sementara selama aplikasi berjalan. Sedangkan localStorage menyimpan data agar tetap ada meskipun halaman di-refresh.

4. Mengapa cleanup diperlukan untuk listener, tetapi tidak untuk localStorage.setItem?

Karena listener harus dihentikan agar tidak terus berjalan dan menumpuk. Sedangkan localStorage.setItem hanya melakukan penyimpanan data satu kali sehingga tidak membutuhkan cleanup.

5. Bagaimana membuktikan navigasi klien dan apa bedanya dengan reload route?

Navigasi klien dapat dilihat melalui DevTools pada bagian Network. Saat berpindah halaman melalui menu, browser tidak perlu memuat ulang dokumen HTML. Sedangkan reload route meminta halaman kembali ke server. Karena itu, hosting perlu diarahkan ke index.html agar halaman tidak mengalami error 404.

Link Github

https://github.com/Noviana910/praktikum-spa-255314083